

Министерство науки и высшего образования Российской Федерации

Пензенский государственный университет

Кафедра «Вычислительная техника»

ОТЧЕТ

по лабораторной работе №9

по курсу «Логика и основы алгоритмизации в инженерных задачах»

на тему «Поиск расстояний в графе»

Выполнили:

студенты групп 24BBB3

Масюк А. Р.

Мамонтов Н. П.

Приняли:

Юрова О. В.

Деев М. В.

Пенза 2025

Цель работы

Изучить и практически реализовать алгоритм поиска расстояний в неориентированном графе на основе модифицированного обхода в ширину с использованием как матрицы смежности, так и списков смежности для представления структуры графа.

Лабораторное задание

Задание:

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа G . Выведите матрицу на экран.
2. Для сгенерированного графа осуществите процедуру поиска расстояний, реализованную в соответствии с приведенным выше описанием. При реализации алгоритма в качестве очереди используйте класс **queue** из стандартной библиотеки C++.

Работа программы

```
input number of verts: 10
0 1 1 0 0 1 0 0 0 0
1 0 0 1 1 1 1 1 1 1
1 0 0 0 1 0 1 0 0 1
0 1 0 0 0 0 1 0 0 1
0 1 1 0 0 1 0 1 0 1
1 1 0 0 1 0 0 1 1 1
0 1 1 1 0 0 0 0 1 1
0 1 0 0 1 1 0 0 0 1
0 1 0 0 0 1 1 0 0 1
0 1 1 1 1 1 1 1 1 0
```

Матрица смежности графа

```
Input start vert: 2
Path: 2 0 4 6 9 1 5 7 3 8
```

Поиск от начальной вершины

```
Distance from 2 to:
0 : 1
1 : 2
2 : 0
3 : 2
4 : 1
5 : 2
6 : 1
7 : 2
8 : 2
9 : 1
```

Подсчет дистанции

Вывод

В ходе выполнения данной лабораторной работы мы изучили и практически реализовали алгоритм поиска расстояний в неориентированном графе на основе модифицированного обхода в ширину с использованием как матрицы смежности, так и списков смежности для представления структуры графа.

Листинг

```
#define _CRT_SECURE_NO_WARNINGS

#include <stdio.h>

#include <stdlib.h>

#include <time.h>

#include <conio.h>

#include <locale.h>

#include <queue>

using namespace std;

void BFSD(int** G, int numG, int* dist, int s) {
    queue<int> q;

    int v;

    q.push(s);

    dist[s] = 0;

    while (!q.empty()) {
        v = q.front();
```

```

        q.pop();

        printf("%3d", v);

        for (int i = 0; i < numG; i++) {
            if (G[v][i] == 1 && dist[i] == -1) {
                q.push(i);
                dist[i] = dist[v] + 1;
            }
        }
    }

    printf("\n\nDistance from %d to: ", s);
    for (int i = 0; i < numG; i++) {
        printf("\n%3d : %3d", i, dist[i]);
    }
}

int main() {
    setlocale(LC_ALL, "Russian");

    int** G;
    int* dist;
    int numG, current;

    printf("input number of verts: ");
    scanf("%d", &numG);

    dist = (int*)malloc(numG * sizeof(int));
    G = (int**)malloc(numG * sizeof(int*));

    for (int i = 0; i < numG; i++) {
        G[i] = (int*)malloc(numG * sizeof(int));
    }

    for (int i = 0; i < numG; i++) {
        dist[i] = -1;

        for (int j = i; j < numG; j++) {
            G[i][j] = G[j][i] = (i == j ? 0 : rand() % 2);
        }
    }
}

```

```

    }
}

for (int i = 0; i < numG; i++) {
    for (int j = 0; j < numG; j++) {
        printf("%3d", G[i][j]);

    }

    printf("\n");
}

printf("Input start vert: ");
scanf("%d", &current);

printf("\nPath: ");

BFSD(G, numG, dist, current);
printf("\n\n");

for (int i = 0; i < numG; i++) {
    free(G[i]);
}

free(G);
free(dist);

__getch();
return 0;
}

```